

Sistema SeS — Resumen Ejecutivo Fase 2



Estado del Proyecto

- Fase 1:**  Completada (Arquitectura base, Auth, DB, Módulos)
- Fase 2:**  Completada (Storage, IA, Realtime, Notificaciones)
- Fase 3:**  Pendiente (Finanzas avanzadas, Reportes, Gráficas)
-



Objetivos de Fase 2 (CUMPLIDOS)

Convertir Sistema SeS en una aplicación SaaS profesional con:

-  Almacenamiento en la nube de archivos multimedia
 -  Inteligencia artificial para análisis de calzado
 -  Notificaciones automáticas inteligentes
 -  Actualizaciones en tiempo real
 -  Sistema de seguimiento completo
-



Funcionalidades Implementadas

1. Supabase Storage (Archivos en la nube)

Antes (Fase 1): Fotos y firmas guardadas como base64 en localStorage (límite 5MB, pérdida de datos al limpiar caché)

Ahora (Fase 2):

- Bucket `fotos` en Supabase Storage con URLs públicas
- Firmas digitales subidas como PNG al servidor
- Galería de fotos migrada completamente a Storage
- Políticas RLS por tenant (cada lavandería solo ve sus archivos)
- Sin límite práctico de almacenamiento
- URLs permanentes que funcionan en cualquier dispositivo

Archivos clave:

- `js/storage-manager.js` — Módulo centralizado para uploads
- `supabase/STORAGE_SETUP.md` — Guía de configuración completa

Impacto: Datos seguros en la nube, accesibles desde cualquier dispositivo, sin riesgo de pérdida.

2. Edge Function para IA (Análisis seguro de imágenes)

Antes (Fase 1): API Key de Claude expuesta en el navegador (riesgo de seguridad crítico), llamadas directas desde el cliente

Ahora (Fase 2):

- Edge Function `analyze-shoe` en servidor de Supabase

- API Key de Anthropic protegida como secret (nunca visible al cliente)
- Análisis completo de calzado: marca, modelo, color, material, estado, tratamiento sugerido
- Confianza del reconocimiento (0-100%)
- Acepta URLs de Storage o base64 directo

Archivos clave:

- `supabase/functions/analyze-shoe/index.ts` — Edge Function completa
- `supabase/functions/DEPLOYMENT.md` — Guía de despliegue paso a paso
- `js/modules/ia.js` — Cliente integrado con Edge Function

Impacto: IA segura, escalable y profesional. Cero riesgo de exponer credenciales.

3. Módulo IA Completamente Funcional

Antes (Fase 1): Placeholder con llamada directa a Claude (insegura)

Ahora (Fase 2):

- Interfaz completa para capturar/subir fotos
- Integración con Edge Function `analyze-shoe`
- Sube fotos a Storage automáticamente si hay orden asociada
- Muestra nivel de confianza del análisis
- Permite editar campos manualmente antes de guardar
- Actualiza automáticamente datos del calzado en la orden

Flujo:

1. Usuario toma foto o sube desde galería
2. Selecciona orden (opcional)
3. Sistema sube foto a Storage (si hay orden)
4. Llama a Edge Function con la URL
5. Claude analiza la imagen (5-15 segundos)
6. Muestra resultados con confianza
7. Usuario revisa/edita y guarda en la orden

Impacto: Digitalización inteligente del proceso de recepción. Reduce errores humanos en identificación.

4. Sistema de Notificaciones Automáticas

Antes (Fase 1): Notificaciones calculadas en memoria, se perdían al recargar

Ahora (Fase 2):

- Notificaciones calculadas automáticamente:
- 📦 Entregas programadas para hoy
- ⚠️ Servicios atrasados
- 📊 Stock bajo en inventario
- 💰 Pagos pendientes/parciales
- Almacenadas en tabla `notificaciones` con RLS
- Auto-sync cada 60 segundos mientras el usuario está activo
- Indicador de campana con contador de notificaciones nuevas

- Botón para descartar notificaciones individualmente
- Prioridad automática (Alta/Media/Baja)

Archivos clave:

- `js/modules/notificaciones.js` — Lógica completa
- `js/db.js` — `createNotification()`, `markNotificationRead()`

Impacto: Los gerentes nunca pierden de vista servicios urgentes. Proactividad automática.

5. Agenda con Supabase Realtime

Antes (Fase 1): Datos estáticos, requiere recargar para ver cambios de otros usuarios

Ahora (Fase 2):

- Suscripción en tiempo real a la tabla `ordenes`
- Actualizaciones instantáneas cuando otro usuario crea/modifica órdenes
- Notificaciones discretas: “Nueva orden agregada”, “Orden actualizada”
- Canal dedicado con filtro por `tenant_id`
- Se activa automáticamente al iniciar sesión
- Se desactiva al cerrar sesión (no consume recursos innecesarios)

Archivos clave:

- `js/modules/agenda.js` — `startRealtimeAgenda()`, `stopRealtimeAgenda()`
- `js/auth.js` — Integración con login/logout

Impacto: Colaboración en tiempo real. Equipos coordinados sin comunicación manual.

6. Timeline y Control de Calidad Sincronizados

Antes (Fase 1): Datos en memoria, sin persistencia real

Ahora (Fase 2):

- **Timeline:** 10 pasos de seguimiento completamente persistidos
- Campos: `timeline_index`, `timeline_dates` (JSONB)
- Fechas de completado guardadas por paso
- Botón “Marcar completado” sincroniza con DB inmediatamente
- **Control de Calidad:** Checklist de 8 puntos
- Campo: `control_calidad` (JSONB)
- Limpieza, Costuras, Pintura, Pegado, Cordones, Plantillas, Fotos, Empaque
- Progreso visible: “5 de 8 verificaciones completadas”
- Solo avanza timeline cuando están todos marcados
- Retrocompatibilidad con datos legacy del HTML original

Archivos clave:

- `js/modules/ordenes.js` — `ensureTimelineFields()`, `advanceTimelineStep()`, `saveCalidadChecklist()`
- `js/db.js` — Campos sincronizados: `timeline_index`, `timeline_dates`, `control_calidad`

Impacto: Trazabilidad completa. Cumplimiento de estándares de calidad verificables.

7. Sistema de Cobros Completo

Antes (Fase 1): Funcional pero sin tracking detallado por método

Ahora (Fase 2):

- Cobro por QR:

- Código QR dinámico con datos de la orden
- Payload: `SISTEMASES|ORDEN:123|MONTO:450.00|CLIENTE:Juan`
- Monto sugerido automático (total pendiente)
- Tracking en campo `pagado_qr`

- Cobro en Efectivo:

- Interfaz simple con monto sugerido
- Tracking en campo `pagado_efectivo`

- Cobro por Tarjeta/Transferencia:

- Campos preparados: `pagado_tarjeta`, `pagado_transferencia`

- Corrección de Pagos:

- Modal exclusivo para administradores
- Permite corregir monto y método sin sumar
- Útil para corregir errores de captura
- Todo sincronizado con DB en tiempo real
- Estado de pago automático: Pendiente → Parcial → Pagado

Archivos clave:

- `js/modules/ordenes.js` — Funciones de cobro completas
- `js/db.js` — Campos `pagado_qr`, `pagado_efectivo`, `pagado_tarjeta`, `pagado_transferencia`

Impacto: Control financiero preciso. Conciliación por método de pago. Transparencia total.



Archivos Nuevos Creados (Fase 2)

```

/home/ubuntu/sneakermania/
├── js/
│   └── storage-manager.js
├── supabase/
│   ├── STORAGE_SETUP.md
│   └── functions/
│       ├── DEPLOYMENT.md
│       ├── DEPLOYMENT.pdf
│       ├── DEPLOYMENT.docx
│       └── analyze-shoe/
│           └── index.ts
└── FASE_2_RESUMEN.md

```

- ← Módulo de uploads a Storage
- ← Guía de configuración de Storage
- ← Guía de despliegue de Edge Functions
- ← Versión PDF
- ← Versión Word
- ← Edge Function completa
- ← Este documento

Archivos Modificados (Fase 2)

js/auth.js	← Integración con notificaciones y Realtime
js/db.js	← Nuevas funciones para notificaciones
js/modules/ordenes.js	← Storage para firmas, timeline persistente
js/modules/galeria.js	← Migración completa a Storage
js/modules/ia.js	← Edge Function integrada
js/modules/notificaciones.js	← Sistema completo de notificaciones
js/modules/agenda.js	← Supabase Realtime
styles/fixes.css	← Estilos para galería mejorada
README.md	← Documentación actualizada con Fase 2

Configuración Requerida para Usar Fase 2

Paso 1: Supabase Storage

1. Ir a Supabase Dashboard → **Storage**
2. Crear bucket **fotos** (marcarlo como público)
3. Ir a **SQL Editor** y ejecutar las políticas RLS de `supabase/STORAGE_SETUP.md`

Paso 2: Edge Function para IA

1. Instalar Supabase CLI:

```
bash
brew install supabase/tap/supabase # macOS
# 0 descargar desde GitHub para Windows
```

2. Obtener API Key de Claude:

- Ir a <https://console.anthropic.com/>
- Crear cuenta o iniciar sesión
- Crear API Key

3. Configurar secret en Supabase:

```
bash
cd /ruta/a/sneakermania
supabase link --project-ref TU_PROJECT_REF
supabase secrets set ANTHROPIC_API_KEY=tu-api-key-de-anthropic
```

4. Desplegar Edge Function:

```
bash
supabase functions deploy analyze-shoe
```

5. Verificar en logs:

```
bash
supabase functions logs analyze-shoe --tail
```

Documentación completa: `supabase/functions/DEPLOYMENT.md`



Estadísticas del Proyecto

Métrica	Valor
Archivos totales	36 archivos
Archivos nuevos (Fase 2)	6 archivos
Archivos modificados (Fase 2)	9 archivos
Líneas de código agregadas	~1,240 líneas
Edge Functions	1 (analyze-shoe)
Buckets de Storage	1 (fotos)
Canales Realtime	1 (agenda-ordenes)
Commits Git	2 commits de Fase 2



Tecnologías Utilizadas (Fase 2)

- **Supabase Storage** — Almacenamiento de archivos en la nube
- **Supabase Edge Functions** — Deno runtime con API de Anthropic
- **Supabase Realtime** — WebSockets para actualizaciones en tiempo real
- **Claude API (Anthropic)** — Análisis de imágenes con IA
- **Canvas API** — Captura de firmas digitales
- **FileReader API** — Procesamiento de imágenes en cliente



Próximos Pasos Recomendados

Inmediato (Antes de usar la app)

1. ☒ Ejecutar migraciones SQL (001, 002, 003) en Supabase SQL Editor
2. ☒ Configurar bucket `fotos` en Storage con políticas RLS
3. ☒ Desplegar Edge Function `analyze-shoe`

Fase 3 (Funcionalidades avanzadas)

1. Alta automática de empleados con cuenta de Auth
2. Reportes con filtros por fecha y exportación a PDF
3. Gráficas de ingresos/gastos con Chart.js
4. Facturación con correlativo legal y validación de RFC

Conclusión

La **Fase 2** transforma Sistema SeS de una aplicación funcional a un **SaaS profesional listo para comercializar**:

- ✓ **Seguridad:** API Keys protegidas en servidor, RLS por tenant
- ✓ **Escalabilidad:** Storage ilimitado, Edge Functions serverless
- ✓ **Inteligencia:** IA para análisis de calzado
- ✓ **Colaboración:** Realtime para equipos distribuidos
- ✓ **Proactividad:** Notificaciones automáticas inteligentes

Estado: Listo para onboarding de primeros clientes.

Desarrollado por: Sistema SeS Team

Fecha: Agosto 2026

Stack: Vanilla JS + Supabase + Claude AI

Licencia: Propietario